# Chapter 2 — AI Engineering Fundamentals

## 2.1 From Deterministic to Probabilistic Software

### Conceptual Deep Dive

Traditional software maps input to output through code you wrote; the same input gives the same output and tests assert equality. An LLM samples tokens from a probability distribution `P(next token | context)`. Even at temperature 0, batching, floating-point non-associativity and provider-side changes make outputs **non-deterministic** in practice. Consequences:

- Tests become **statistical**: run N samples, assert a pass rate.
- Correctness is a **spectrum** (faithful, relevant, safe, well-formatted), so you build graded evals, not boolean asserts.
- Behaviour **drifts** when the provider updates a model alias.
- This non-determinism is inherent, not a bug to be eliminated.
- Failure modes are semantic (confident wrong answers), not exceptions.

Sampling controls: **temperature** rescales logits (`softmax(z/T)`); low T sharpens the distribution, high T flattens it. **top-p** keeps the smallest set of tokens with cumulative probability ≥ p. **top-k** keeps the k most likely. **Seed** helps reproducibility where supported but is not a guarantee.

### Cross-Connection Scenarios

Because outputs vary, Chapter 7 (validation/retries) and Chapter 10 (statistical evals) exist. Chapter 15 turns eval pass-rates into CI gates.

### Production Anti-Patterns

- `assert response == "expected"` tests that flake.
- Pinning to a floating alias (`latest`) and being surprised by regressions.
- Parsing free text with regex instead of constraining output.

### Minimal Working Code

```
import math
import random

def softmax(logits, temperature=1.0):
    scaled = [z / max(temperature, 1e-6) for z in logits]
    m = max(scaled)
    exps = [math.exp(z - m) for z in scaled]
    s = sum(exps)
    return [e / s for e in exps]

def sample(probs, top_p=1.0, rng=random):
    order = sorted(range(len(probs)), key=lambda i: -probs[i])
    kept, cum = [], 0.0
    for i in order:
        kept.append(i)
        cum += probs[i]
        if cum >= top_p:
            break
```

```
    weights = [probs[i] for i in kept]
    return rng.choices(kept, weights=weights)[0]

logits = [2.0, 1.0, 0.1]
print(softmax(logits, 0.2), softmax(logits, 2.0))
print(sample(softmax(logits), top_p=0.9))
```

# 2.2 The Modern AI Stack

## Conceptual Deep Dive

Layers, bottom to top: **hardware** (GPU/TPU) → **runtime/serving** (vLLM, TGI) → **models** (closed API or open weights) → **orchestration** (prompts, tools, agents, RAG) → **data** (vector stores, feature stores, caches) → **evaluation & observability** → **application/UX**. Decisions at each layer trade cost, latency, quality, privacy and control.

Key budgets: **context window** (max tokens in+out), **latency** (time-to-first-token and tokens/sec), **cost** (price per million input/output tokens; output usually costs more), **throughput** (requests/sec under GPU memory constraints).

## Cross-Connection Scenarios

Model choice (closed vs open) links to fine-tuning (12), serving (13) and security/data residency (14). Context window limits drive chunking and retrieval (9).

## Production Anti-Patterns

- Using the largest model for every call; route easy tasks to small models.
- Stuffing the entire corpus into context ("long context replaces RAG") without measuring cost and lost-in-the-middle degradation.
- No fallback provider.

## Minimal Working Code

A model router keyed on task difficulty:

```
from dataclasses import dataclass

@dataclass(frozen=True)
class ModelSpec:
    name: str
    cost_per_mtok_in: float
    cost_per_mtok_out: float
    max_context: int

SMALL = ModelSpec("small-model", 0.1, 0.4, 128_000)
LARGE = ModelSpec("large-model", 3.0, 15.0, 200_000)

def route(task: str, prompt_tokens: int) -> ModelSpec:
    hard = any(k in task for k in ("reason", "plan", "code", "legal&qu
ot;))
    if prompt_tokens > SMALL.max_context or hard:
        return LARGE
    return SMALL

def estimate_cost(m: ModelSpec, tin: int, tout: int) -> float:
    return tin / 1e6 * m.cost_per_mtok_in + tout / 1e6 * m.cost_per_mtok_out

m = route("summarize", 2_000)
print(m.name, estimate_cost(m, 2_000, 300))
```

# 2.3 Developer Mental Models

## Conceptual Deep Dive

1. **The model is a function of its context.** It has no memory beyond what you send. "Memory" is something you build (Chapter 11).
2. **Prompts are programs; evals are tests.** Version both.
3. **Capability ladder:** prompt → few-shot → RAG → tools → fine-tune → train. Escalate only on eval evidence.
4. **Cost-latency-quality triangle:** you can optimise two; caching, smaller models and batching shift the frontier.
5. **Data flywheel:** logs → failures → eval cases → improvements.

## Cross-Connection Scenarios

The flywheel connects observability (14) → evaluation (10) → CI (15). The capability ladder maps to chapters 6 → 9 → 11 → 12.

## Production Anti-Patterns

- Jumping to fine-tuning to fix a retrieval problem.
- Believing the model "knows" your private data.
- Shipping without a way to capture failing examples.

## Minimal Working Code

```
import json
import time
from pathlib import Path

LOG = Path("interactions.jsonl")

def log_interaction(prompt: str, output: str, meta: dict | None = None) -> None:
    rec = {"ts": time.time(), "prompt": prompt, "output": output, **(meta o
r {})}
    with LOG.open("a", encoding="utf-8") as f:
        f.write(json.dumps(rec) + "\n")

def promote_to_eval(rec_index: int, expected: str, out: Path = Path("eval_cases.jsonl")) ->
None:
    recs = [json.loads(l) for l in LOG.read_text(encoding="utf-8").splitlines()]
    case = {"input": recs[rec_index]["prompt"], "expected": expected}
    with out.open("a", encoding="utf-8") as f:
        f.write(json.dumps(case) + "\n")
```

# 2.4 Token Economics and Latency

## Conceptual Deep Dive

Tokens, not characters, are billed. English averages ~0.75 words/token; code and non-Latin scripts are costlier. Latency = queueing + prefill (processes the prompt in parallel, compute-bound) + decode (one token at a time, memory-bandwidth-bound). TTFT is dominated by prefill; inter-token latency by decode. **Prompt caching** reuses KV state for repeated prefixes, cutting cost and TTFT.

## Cross-Connection Scenarios

Streaming (4) improves perceived latency; KV cache (5) and continuous batching (13) explain provider-side behaviour.

## Production Anti-Patterns

- Putting volatile data at the start of the prompt, destroying prefix caching.
- Unbounded max_tokens.
- Ignoring output tokens in cost models.

## Minimal Working Code

```
def budget(requests_per_day: int, tin: int, tout: int, p_in: float, p_
out: float) -> float:
    per_req = tin / 1e6 * p_in + tout / 1e6 * p_out
    return per_req * requests_per_day * 30

print(f"${budget(50_000, 1500, 400, 0.5, 2.0):,.2f}/month")
```

# 2.5 Failure Taxonomy of LLM Systems

## Conceptual Deep Dive

Classify failures to fix the right layer: **knowledge** (stale/missing facts), **reasoning** (multi-step slips, arithmetic), **instruction** (ignored constraints, format drift), **retrieval** (wrong/missing context), **tooling** (bad arguments, wrong tool), **safety** (injection, harmful output), **systems** (timeouts, rate limits, truncation) and **UX** (ambiguous queries, missing clarification). Edge cases: silent truncation when input exceeds the context window; finish_reason == "length" producing cut-off JSON; refusals that look like answers; sycophancy (agreeing with a wrong user premise); positional bias; and "format collapse" in long outputs. Tagging each failed eval case with one category turns a pile of anecdotes into a prioritised backlog.

## Cross-Connection Scenarios

Retrieval failures route to Chapters 8–9; instruction failures to 6–7; safety to 14; systems to 4 and 13.

## Production Anti-Patterns

- Responding to every failure by editing the prompt.
- Ignoring finish_reason and parsing truncated output.
- No tagging of failures, so no way to prioritise.

## Minimal Working Code

```
from collections import Counter

CATEGORIES = {"knowledge", "reasoning", "instruction", "retrieval"
;, "tooling", "safety", "systems", "ux"}

def triage(failures: list[dict]) -> list[tuple[str, int]]:
    bad = [f for f in failures if f["category"] not in CATEGORIES]
    if bad:
        raise ValueError(f"unknown categories: {bad[:2]}")
    return Counter(f["category"] for f in failures).most_common()

print(triage([{"category": "retrieval"}, {"category": "retrieval"
}, {"category": "instruction"}]))
```

```
def check_finish(resp: dict) -> str:
    """Never parse a response that was cut off."""
    reason = resp.get("finish_reason")
    if reason == "length":
        raise ValueError("output truncated: raise max_tokens or shorten the task")
    if reason == "content_filter":
        raise PermissionError("provider filtered the response")
    return resp["text"]

print(check_finish({"finish_reason": "stop", "text": "ok"}))
```

# 2.6 Build vs Buy and Vendor Strategy

## Conceptual Deep Dive

Decide per layer: use a **hosted API** for fastest time to value and frontier quality; **managed open-model hosting** for cost with less ops; **self-hosting** for data control, latency, or volume. Mitigate lock-in with a thin provider-agnostic interface, prompts that avoid vendor quirks, portable eval suites, and a tested secondary provider. Consider data-processing terms (retention, training on your data), regional availability, rate-limit tiers, deprecation policies, and SLAs. Edge cases: a provider retiring a model with 3–6 months' notice, silent quality changes behind an alias, and regional outages.

### Cross-Connection Scenarios

Gateways (13.6), fallbacks (4.4), fine-tuned replacements (12, 17.5), legal/security review (14.5).

### Production Anti-Patterns

- Spreading vendor SDK types through the whole codebase.
- Having a "backup provider" that was never load-tested.
- Ignoring deprecation announcements until the cutoff date.

### Minimal Working Code

```
from typing import Protocol

class ChatModel(Protocol):
    def chat(self, messages: list[dict], **kw) -> str: ...

class ProviderA:
    def chat(self, messages, **kw):
        return "from A"

class ProviderB:
    def chat(self, messages, **kw):
        return "from B"

def chat_with_fallback(models: list[ChatModel], messages):
    last = None
    for m in models:
        try:
            return m.chat(messages)
        except Exception as e:
            last = e
    raise RuntimeError("all providers failed") from last

print(chat_with_fallback([ProviderA(), ProviderB()], [{"role": "user", "content&
quot;: "hi"}]))
```

```
from datetime import date

DEPRECATIONS = {"old-model": date(2026, 12, 31)}

def deprecation_warnings(used: list[str], today: date, warn_days: int = 120):
    return [f"{m} retires {d} ({(d - today).days}d)" for m, d in DEPRECATIONS.items()
            if m in used and (d - today).days <= warn_days]

print(deprecation_warnings(["old-model"], date(2026, 10, 22)))
```

## 2.7 The Model Landscape: Closed, Open, Reasoning, Small and Multimodal

### Conceptual Deep Dive

Model categories matter more than names. **Frontier closed models** lead on hard reasoning, coding and long-horizon agents; they cost the most and give least control. **Open-weight models** (Llama, Qwen, Mistral, Gemma, DeepSeek families) let you self-host,

fine-tune and inspect, with licences ranging from permissive to restricted. **Reasoning models** spend hidden "thinking" tokens before answering, improving maths, code and planning at higher latency and cost; they usually need less explicit chain-of-thought prompting and often reject unnecessary instructions about format of reasoning. **Small language models (SLMs, 1–8B)** run on laptops or cheap GPUs and excel when fine-tuned for narrow tasks. **Mixture-of-Experts** models give large capacity at lower active-parameter cost but need large memory. **Multimodal models** accept images, audio or video. **Embedding and reranker models** are separate specialised families. Selection method: shortlist 3 models per tier, run your eval set, plot quality vs cost vs latency, and pick the Pareto-optimal option with a fallback. Edge cases: aliases that silently move, context windows advertised vs effective, tokenizers producing different costs for the same text, and regional availability.

## Cross-Connection Scenarios

Routing and cascades (13.6), fine-tuning small models (12, 17.5), eval-based selection (10.6), token economics (2.4).

## Production Anti-Patterns

- Choosing the newest model because it tops a public leaderboard.
- Using a reasoning model for simple classification (cost and latency waste).
- Treating advertised context length as usable recall length.

## Minimal Working Code

```
def pareto_front(candidates: list[dict]) -> list[dict]:
    """Keep models not dominated on (quality up, cost down, latency down)."""&quo
t;
    front = []
    for a in candidates:
        dominated = any(
            b is not a and b["quality"] >= a["quality"] and b["cost"] &l
t;= a["cost"]
            and b["latency"] <= a["latency"]
            and (b["quality"] > a["quality"] or b["cost"] < a["c
ost"] or b["latency"] < a["latency"])
            for b in candidates)
        if not dominated:
            front.append(a)
    return front

models = [
    {"name": "frontier", "quality": 0.92, "cost": 10.0, "lat
ency": 3.0},
    {"name": "mid", "quality": 0.88, "cost": 2.0, "latency&q
uot;: 1.5},
    {"name": "small-ft", "quality": 0.87, "cost": 0.2, "late
ncy": 0.4},
    {"name": "old-mid", "quality": 0.80, "cost": 2.5, "laten
cy": 2.0},
]
print([m["name"] for m in pareto_front(models)])
```

```
def effective_context_probe(llm, filler_tokens: int, depth: float, nee
dle="The access code is 7391."):
    """Needle-in-haystack: bury a fact at a relative depth and test recall."""&qu
ot;
    words = ["lorem"] * filler_tokens
    pos = int(depth * len(words))
    words.insert(pos, needle)
    answer = llm(" ".join(words) + "\n\nWhat is the access code?")
    return "7391" in answer

# sweep depth in (0.0, 0.25, 0.5, 0.75, 1.0) and length in several sizes; plot the recall grid
```

# 2.8 AI Product Patterns and Human-AI Interaction Design

## Conceptual Deep Dive

Product shape determines engineering requirements. Patterns: **chat assistant** (open dialogue; needs memory, guardrails, streaming), **copilot/inline suggestion** (low latency, accept/reject telemetry), **form-filling/extraction** (structured output, review UI), **search and answer** (RAG with citations), **workflow automation** (deterministic graph with LLM steps), **autonomous agent** (long-running, approvals, budgets), **batch processing** (offline, cost-optimised). Interaction design principles: set expectations early, show sources and confidence, make correction cheap (edit, regenerate, thumbs), support undo, ask clarifying questions when ambiguity is high instead of guessing, degrade gracefully (partial results, "I'm not sure"), and preserve user control for consequential actions. Instrument acceptance rate, edit distance, time saved, and abandonment. Edge cases: latency tolerance differs by pattern (inline completion under 300 ms, deep research minutes), trust calibration (over-trust vs under-trust), and accessibility.

## Cross-Connection Scenarios

Metrics translation (16.1), feedback-driven eval datasets (10.1, 2.3), streaming UX (4.2), approvals (7.7, 11.8).

## Production Anti-Patterns

- Using a chat box for every problem when a form or button would be faster and safer.
- No feedback controls, so no learning signal.
- Hiding sources, preventing users from verifying claims.

## Minimal Working Code

```
def pick_pattern(req: dict) -> str:
    if req.get("offline") and req.get("volume") == "high":
        return "batch processing"
    if req.get("latency_ms", 1000) < 400:
        return "inline copilot (small/fast model, aggressive caching)"
    if req.get("structured_target"):
        return "form-filling/extraction with review UI"
    if req.get("needs_knowledge"):
        return "search-and-answer (RAG with citations)"
    if req.get("steps_known"):
        return "workflow automation"
    return "chat assistant or agent with approvals"

print(pick_pattern({"latency_ms": 250}))
```

```
from collections import Counter

def feedback_summary(events: list[dict]) -> dict:
    c = Counter(e["type"] for e in events)
    shown = c["shown"] or 1
    return {"accept_rate": c["accept"] / shown, "regen_rate": c["regen
erate"] / shown,
            "thumbs_down_rate": c["thumbs_down"] / shown}

print(feedback_summary([{"type": "shown"}] * 10 + [{"type": "accept&qu
ot;}] * 6 + [{"type": "regenerate"}] * 2))
```